M. Aditya Vardhan
2420090094
S-10

# CO-3 Assignment

1. Explain the architecture of spring boot MVC and the complete flow of web application

A **Architecture Components :-**

-> Client [Browser] -> sends request

-> Dispatcher servlet -> Central request handler

-> Controller layer -> handles request

-> Service layer -> handles request

-> Repository layer -> business logic

-> Database

**Flow :-**

-> Client sends HTTP request

-> Dispatch servlet recieves it

-> It maps requests to appropiate controller

-> controller calls service layer

-> Repository interacts with DB

-> Responce return back through some layers.

2) Explain annotations @ Controller, @ RestController @ GetMapping with example

A **@ Controller :-**

* used for spring MVC [web pages]
* Returns view name

```
@Controller
    Public class Controller {
        @Get Mapping ("/home")
```

```
Public string home(){
        return "home";
    }
}
```

@ Rest Controller :-

- combination of [@ Controller + @ Response body]
- Return JSON data directly

```
@ Rest controller
    Public class user controller {
        @ Get Mapping ("/user")
            Public string get user(){
                return "Hello user";
            }
        }
```

@ Post Mapping :-

- Handles HTTP POST request

```
@ Post Mapping ("/users")
    Public user add user (@ Request user) {
        return userservices save user (user);
    }
```

3) Explain the purpose of @ Request Body and @ Response body in handling JSON data for REST API's

A @ Request Body :-

- Converts JSON -> Java project

```
@ Post Mapping ("/users")
    Public user save user (@ Request body user) {
        return user;
    }
```

JSON Input :-

```
{
"name" : "John",
"age" : 25,
}
```

Response Body :-

* Converts Java objects → JSON

```
@GetMapping ("user")
@ResponseBody
Public user getUser() {
        return new user ("John", 25);
}
```

4) Describe the repository layer, DAO & service layers in Spring Data JPA

A Repository layer :-

→ Interface that interacts with DB
→ Extends JPa Repository

~~Public~~ DAO layer

→ Traditional DB handling layer
→ In Spring Boot, replaced by Reposity

Service layer :-

→ Contains bussiness logic

```
@Service
Public class user service {
    @Auto wired
    Private user Repository repo;
```

```
Public list <user> get All users() {
        return repo find All ();
    }
}
```

5. Explain different types of web services

A 1. SOAP [simple object Access Portal]:-

-> XML Based

-> Strick standards

2. REST [Representations State Transfer]:-

-> user HTTP methods

-> JSON /XML

-> light weight & fast

3. Graph QL

-> client specificey require data

-> Flexible queires

6) Describe how to build REST APT in spring Boot of CRUD operation

A Entity :-

```
@ Entity
Public class User {
    @ Id
    @ Generated value
    Private long id;
```

```
Private String name;
}
```

Repository :-

```
-> Public interface User Repository extends Spa Repistory
   <user>, <long> {}
```

Controller :-

```
@Rest Controller
@ Request Mapping ("/users")
Public class user controller {
    @ Auto wired
    Private user repository repo;
    @ Post Mapping
    Public user Create (@ Request Body user user) {
        return reposave (user);
    }
    @ Get Mapping
    Public List <user> read() {
        return repo find ALL ();
    }
```

7. Explain JPQL with examples using @ Query

A JPQL = Java Persistence Query language

-> works with entities (not tables)

Ex-1 :

```
@Query ("SELECT FROM User "WHERE u.name="name")
List<User> findByName (@Param("name") String name)
```

Ex 2 :-

```
@Query ("SELECT u.From User WHERE u.age > 20"
List<User> find User Above Age();
```

Find by :-

```
List <User> findby Name (String name);
```

delete by :-

```
Void delete by Name (String Name);
```

Update by :-

```
@Modifying
@Query ("Update user a SET u.name = :name
        WHERE u.id = :id)
Void upadete user (@Param ("id") long id, @Param ("name")
        String name);
```

---